

Upgrade Flight Checklist

Three phases, three gates. Challenge on the left, response on the right – read it out and answer it honestly.

This merges your draft with the three checklists already in the deck (slides 23, 54 and 76), the ordering slide (55) and the Go / No-Go gate (51). Where the two disagreed, the deck won and the difference is explained at the bottom.

A checklist is not a to-do list. Every line is a question somebody answers out loud, and **“no” is a valid answer** – as long as it is written down, with a reason and a date. That is the difference between an accepted risk and a forgotten one.

☐

unmarked – from your draft

☐

added – missing from the draft

☐

moved or corrected

PHASE 1

Dispatch & Pre-Flight

Nothing here touches the target version. The whole aim is to reach the runway with a known aircraft, a read route and a working set of instruments.

The aircraft you have

☐	Current version	known, on its latest patch, documented	
☐	Own extensions	inventoried – and each one has a named owner	☐
☐	Third-party extensions	a target-version release exists, or a plan does	
☐	Unowned extensions	forked, replaced or dropped – decided now, not in flight	☐

☐	Your changes to them	Fluid overrides, FlexForm extensions, XCLASSes, hooks – listed	!
--------------------------	-----------------------------	--	---

☐	Platform	PHP, database, web server, image processing – versions recorded	
--------------------------	-----------------	---	--

The route ahead

☐	Target version	decided, dated and justified against the support calendar	+
--------------------------	-----------------------	---	---

☐	Changelogs	read and triaged – Breaking, Deprecation, Feature, Important	
--------------------------	-------------------	--	--

☐	Breaking changes	the ones that touch your code, extracted into a work list	+
--------------------------	-------------------------	---	---

☐	PHP baseline	one version supported by both source and target – and live in production before departure	!
--------------------------	---------------------	---	---

Why the baseline moves first. If PHP and the core change in the same step, every failure has two possible causes. Move production onto the overlap version while you are still on the old core, and the flight has one variable instead of two.

Airworthy on the version you are on

☐	Deprecations	cleared on the current core, or on the MEL with a reason	
--------------------------	---------------------	--	--

☐	Extension Scanner	run – strong matches at zero	
--------------------------	--------------------------	------------------------------	--

☐	TCA migrations	checked in the backend module – the list is empty	+
--------------------------	-----------------------	---	---

☐	Rector & Fractor	clean against the current version's rule set	
--------------------------	-----------------------------	--	--

☐	Deprecation log	switched on and read under real editor traffic	+
--------------------------	------------------------	--	---

☐	CI	scanner and Rector wired in, so none of this can drift back	+
--------------------------	-----------	---	---

Two of these have no command line. The TCA migration check is a backend module only. And the Extension Scanner has no rule for some deprecations at all — `QueryBuilder->execute()` is invisible to it. Rector finds that one. Neither tool alone is the answer.

Instruments

☐	A test suite exists	it runs, and it executes the code that touches the core	+
☐	Coverage	aimed at core contact points, not at a percentage	+
☐	Baseline recorded	the suite's current result is known and understood	+

This is the one your draft was missing entirely. Phase 2 says “run the toolchain until it is green” — which quietly assumes a suite that only Phase 1 can build. Every line your tests execute is a line the core can warn you about; every line they do not is a line you will meet in production.

Crew and contingency

☐	Briefing	written — what changes, what to expect, who does what	
☐	QRH	the failures you expect, each with its prepared answer	+
☐	MEL	what you accept as broken — reason and rectification date each	+
☐	Backup and restore	rehearsed, not assumed	+
☐	Rollback criteria	written down before departure, not invented during it	+
☐	Window and people	long enough for the plan plus surprises, with the right people present	+

GATE — GO / NO-GO

Phase 1 ends in a decision, not in a feeling. No-Go is a professional outcome: delaying a flight costs money, and the alternative costs more.

GO

- Strong matches at zero, or on the MEL with a date
- Every third-party extension resolved
- A backup restored successfully at least once
- Rollback criteria written and understood by everyone
- The window fits the plan plus surprises

NO - GO

- "We will find out during the upgrade"
- An unowned extension nobody can update
- A backup that has never been restored
- A window that fits only the happy path
- The person who knows the system is on holiday

PHASE 2

Take-Off & Cruise

Here the order is not a preference. Do these in sequence, or do several of them twice.

Before the roll

☐	Freeze announced	content, code and deploys — everyone knows	+
☐	Backup taken	database and files, verified — not just triggered	+

The sequence

☐	1 Branch	everything after this point is reversible
☐	2 Platform	PHP version, composer platform config, extension constraints
☐	3 Packages	core, extensions and tooling raised in one transaction

☐	4	Rector	after the packages – reviewed hunk by hunk	
☐	5	Fractor	after the packages – TypoScript, Fluid, YAML, the half Rector never sees	
☐	6	Scan	Extension Scanner <i>and</i> the TCA check, both now carrying target rules	
☐	7	Schema	extension setup and the database compare – the point of no return	
☐	8	Wizards	list them, run them, then verify what each one actually did	
☐	9	Caches	flush, warm, and only then look at the site	

Your draft ran the wizards third. They belong at eight. A wizard migrates data against the new schema, and the schema can only be built once the new code is on disk. Run them early and you migrate data into a shape the core has not created yet. The same trap catches Rector: run it before step 3 and it refactors against the old rule set.

Working the failures

☐	One at a time	red tests worked in order, not in parallel panic	
☐	Generated code read	anything Rector scaffolded is reviewed before it is trusted	
☐	Third-party gaps	closed by update, patch, fork or replacement	
☐	Your overrides	Fluid templates and FlexForms re-based onto the new originals	
☐	New deprecations	target-version warnings cleaned as they appear, not deferred	
☐	Two kinds of error	separated – broken <i>by</i> the upgrade, or broken already	

Rector is a co-pilot, not an autopilot. In the demo project it migrated the plugin registration correctly and scaffolded an upgrade wizard containing `// TODO: Add this mapping yourself!` – which, left as written, silently orphans every existing content record. Nothing failed. No scan complained. Only the test that renders the plugin in the frontend noticed.

Cockpit discipline

☐	Sterile cockpit	no unrelated refactoring in this branch	☐
☐	One tool, one commit	so the diff stays reviewable by somebody else	☐
☐	Go-around criteria	when to stop and reset – agreed in advance	☐

GATE – CLEARED TO HAND OVER

Green tooling is necessary and not sufficient. A human still has to use the thing.

CLEARED

- CGL, static analysis, unit and functional suites all green
- Frontend and backend smoke-tested by a person
- Deployed to staging on production-like data
- Remaining scanner findings recorded and accepted

NOT YET

- Tests green because they were skipped or weakened
- Generated wizards nobody has read
- “It works locally”
- Editors have not touched it

PHASE 3

Approach, Landing & Debrief

Green again is not the end of the work. It is the end of the flight — and the start of the preparation for the next one.

Straight after landing

☐	Frontend	smoke-tested by a human, not by a status code	+
☐	Backend	editors can do their actual daily work	+
☐	Scheduler	tasks registered, running, and finishing	+
☐	Logs	error rates watched for the first days, not the first hour	+

Sweep again, with the new rules

☐	Extension Scanner	re-run — it now warns about the version after this one	
☐	TCA migrations	re-checked — the list changes with the new core	+
☐	Deprecation log	enabled on the new version and read under real traffic	!
☐	What static analysis missed	collected from the log and turned into work items	

The log catches what the scanners structurally cannot. String class names, variable method calls and \$GLOBALS juggling are invisible to static analysis. They are perfectly visible to a running site with real editors on it.

Remove the ferry equipment

☐	Shims and polyfills	the ones added “just for the migration”, now gone	+
☐	Upgrade-only packages	removed, or moved into an isolated tools install	+
☐	Resolved markers	the todo comments you actually fixed — deleted, not left to rot	+
☐	Constraints	re-pinned to the new reality in your own extensions	+

☐	CI	moved to the new core; rule sets raised to the <i>next</i> target	+
--------------------------	-----------	---	--

Record it while you still remember

☐	New features	triaged – adopt now, schedule, or decline with a reason	
☐	Upgrade report	predicted versus actual, and what took longest	
☐	Documentation	updated now, not “when things calm down”	+
☐	This checklist	surprises become new lines; unused lines get deleted	+
☐	Maintenance plan	the next flight dated from the support calendar	

And then Phase 3 becomes Phase 1. Recording the new deprecations is not the end of this upgrade – it is the preparation for the next one. Teams for whom upgrades are painless are not luckier. They simply never left Phase 3.

What changed in your draft, and why

The three phases were right and the aviation framing held up throughout. Four things were genuinely wrong or missing; the rest was sharpening.

The upgrade wizards were in the wrong place

CORRECTED — SEQUENCE

Your Phase 2 ran wizards before Rector, Fractor and the scanner. A wizard migrates data against the new database schema, so it has to come *after* the schema is built, which in turn needs the new code on disk. Wizards moved to step 8, behind the schema update at step 7. Rector and Fractor sit at 4 and 5 for the mirror-image reason: run either before the packages move and it refactors your code against the rule set you are leaving.

Phase 1 had no instruments

ADDED — MISSING PREREQUISITE

Phase 2 said “check the toolchain until it passes green”, but nothing in Phase 1 established that a test suite exists or that it touches the core. That is the assumption the whole flight rests on: the tests are what convert an invisible behaviour change into a red line you can act on. Added as its own group, including the point that coverage should be aimed at core contact points rather than at a number.

The schema step was missing entirely

ADDED — STEP 7

Between the tooling and the wizards sits the database compare. It is also the one step that is genuinely hard to walk back, which is worth naming out loud on a checklist that otherwise promises reversibility.

No TCA migration check anywhere

ADDED — PHASES 1, 2 AND 3

The core silently migrates outdated TCA on every request and tells you only in a backend module. There is no CLI command for it, so it is easy to leave out of exactly this kind of list — and the message list is not stable across versions, which is why it appears in all three phases rather than once.

The PHP baseline needed one more word

SHARPENED

“One baseline throughout” was the right instinct. Made explicit: pick the version supported by *both* the source and target core, and get production onto it *before* departure. Then the flight changes one variable instead of two, and every failure has one candidate cause.

Nothing decided whether to fly

ADDED — THREE GATES

The draft flowed from phase to phase without a decision point. Each phase now ends in an explicit gate, and Phase 1’s is a real Go / No-Go with No-Go criteria written before anyone is under pressure. Backup-and-restore, rollback criteria and the MEL were added for the same reason — they are what a gate actually checks.

Smaller additions

ADDED

Freeze announcement and a verified backup before the roll; cache flush after the wizards; scheduler tasks and log watching after landing; removing the ferry equipment; re-pinning constraints and raising the rule sets to the *next* target so Phase 3 feeds Phase 1. Your item about adapting third-party extensions you had extended moved to Phase 1 as discovery — you want that list before departure, not while a test is red.